Task 1. Install Jenkins and verify the dashboard

1. Start Jenkins

```
docker volume create jenkins_home

docker run -d \
  --name jenkins \
  -p 8080:8080 \
  -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  jenkins/jenkins:lts
```

2. Get the initial administrator password

```
docker exec jenkins cat
/var/jenkins_home/secrets/initialAdminPassword
```

3. Open the dashboard

Visit http://localhost:8080 in your browser, enter the password, install the suggested plugins, and create your administrator account.

Task 2. Create the freestyle job SpotifyPlaylistBuild

1. In Jenkins, select New Item.
2. Enter SpotifyPlaylistBuild.
3. Select Freestyle project and click OK.
4. Under Build Steps, select Execute shell (Linux-based Jenkins).
5. Enter:
   - echo "Building Spotify Playlist App"
6. Click Save, then Build Now.
7. Open the build number and select Console Output.

Task 3. Create a basic Jenkins pipeline

Create a new item named SpotifyPlaylistPipeline, choose Pipeline, and enter this script in the Pipeline script editor.

```
pipeline {
    agent any
```

```
    stages {
        stage('Checkout') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/octocat/Hello-World.git'
            }
        }

        stage('Build') {
            steps {
                echo 'Building Spotify Playlist App'
            }
        }
    }
}
```

Task 4. Add a test stage and make failures visible

```
    pipeline {
        agent any

        stages {
            stage('Checkout') {
                steps {
                    git branch: 'main',
                        url: 'https://github.com/python/cpython.git'
                }
            }

            stage('Build') {
                steps {
                    echo 'Build completed successfully'
                }
            }
```

```
        stage('Test') {
            steps {
                sh 'python3 --version'
                sh 'exit 1'
            }
        }
    }
}
```

For a real application, replace it with the actual test command, such as:

```
python3 -m unittest discover -s tests
```

Task 5. AI-generated Jenkinsfile for a Zomato-style application

Create a simple Node.js food-ordering application with a package.json and a test.js file.

- package.json

```
{
  "name": "zomato-food-app",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "build": "node --check app.js",
    "test": "node test.js"
  }
}
```

- app.js

```
const message = "Welcome to Zomato Food Ordering App";
console.log(message);
module.exports = { message };
```

- test.js

```
const assert = require("node:assert/strict");
const { message } = require("./app");
```

assert.equal(message, "Welcome to Zomato Food Ordering App");
console.log("All tests passed");

- Jenkinsfile